Principal-Safe Local-First State & Execution Binding

What this initiative is, why it exists, the decisions it locked, what it costs, and the evidence discipline that gates every step of its rollout.

1 · The bug that started it

A conversation displayed the Codex icon while its model label said Fable. That was an accurate rendering of a real split: the conversation was *requested* as Codex, but its first message was delivered to a fallback Claude session, while a later message reached the originally created Codex thread. One logical conversation, two runtimes, both convinced they own it. Agent identity was being inferred from competing in-memory maps in the daemon, and delivery raced session creation.

Investigating it surfaced the deeper question: codecast is built on Convex — a reactive database whose whole premise is that state converges — yet the client keeps drifting from the server and needs repair loops to recover. The same root shape shows up as separate incidents we each keep fixing one at a time: the stale-draft bug, the dispatch black hole ("message hasn't reached the agent"), ghost rows after deletion, and optimistic sends that vanish from the UI while the server says `delivered`. Each fix has historically added another crawl, sweep, timer, or merge rule. The repair code is now load-bearing, and every new feature has to thread through it.

> **The claim this initiative makes:** these are one bug, and it lives in the state model — several stores (inboxStore, idbCache, outbox, change feed) each partially authoritative, with implicit rules for who wins. The fix is to make authority explicit once, so the repair machinery can be deleted rather than extended.

2 · The whole design in one paragraph

Convex remains the only authority on facts and access — that does not change. The local database becomes a *materialized read model*: server state enters it through one small typed apply boundary that knows exactly what each result proves (a complete view, a bounded page, or a proven delta — each with distinct, declared semantics for what omission means). User intent is journaled durably as commands with idempotency keys, projected optimistically, and reconciled by explicit server receipts — never by value-matching. UI reads pure selectors over the local store; it is instant and works offline, and it never runs repair procedures. Separately, one execution coordinator binds each conversation epoch to exactly one runtime, and message delivery consumes that binding instead of guessing from in-memory maps. All local state is principal-scoped: logout purges it, account-switch fences it, and one principal's rows cannot leak into another's session.

3 · Principles — and what we deliberately refused to build

The central discipline, quoted from the design doc:

> **"Infrastructure may centralize a rule, but it may not invent a second domain model."**
> And its success test: *"The architecture is succeeding only if feature code shrinks and repair code is deleted."*

The first implementation was rejected and fully deleted. It tried a broad cross-entity replication system and grew relationship-aware: special by-ID comment paths, conversation-dependent invalidations, scope metadata copied into client rows, a very large feed hook. It was technically ambitious and architecturally worse — replication had started reconstructing domain relationships and access control. Killing it produced the constraints the current design is built on.

Non-goals (permanent)

- Replacing Convex with a client-owned authority
- A universal relation-aware replication graph
- Client-side derivation of authorization — the client may cache an access result, never mint one
- A global ordered change feed for every collection
- Event sourcing for product state
- Forcing every screen off direct Convex queries — demand-driven views stay ordinary reactive queries

Standing red flags (auto-reject)

- A generic feed hook with per-entity branches
- Cached row presence treated as authorization
- Omission from a page treated as deletion
- Acknowledgment inferred from matching values
- A runtime fallback choosing another agent family
- A second runtime registry that delivery trusts

4 · Key decisions, and why

DECISION	RATIONALE
Server-issued view revisions (`local_view_heads`) instead of timestamps or change feeds	Every domain write advances one counter per (principal, view); every query result carries it as coverage. Staleness becomes a comparison, and `Date.now()` stops masquerading as a commit cursor.
Complete-view replacement as the default sync shape	A result whose contract proves "this is the whole membership" can replace the view atomically. Deletion works with zero tombstone machinery — absence from a complete view *is* the deletion. Pages and windows never gain that authority.
Durable command receipts (`local_command_receipts`) with argument fingerprints	Retry-safe by construction: a command id is bound to a SHA-256 of its canonical arguments; replays return the receipt, reuse with different intent is rejected. No payload retained server-side.
Per-principal local stores with generation fencing	Account switching and logout were correctness holes (protected rows outliving the session). Now each principal gets its own store; a superseded generation is fenced at the storage layer and cannot write.
One writer boundary per synced table, enforced by CI	Guard tests fail if any module inserts `comments` / `bookmarks` by literal table name — the revision-aware writer is structurally the only path, so coverage can't silently rot as the codebase grows.
Execution: epoch-fenced bindings (4 tables), off by default	One binding per conversation epoch, delivery permits that cannot cross epochs or device owners, and a boot registry for orphan recovery. The Codex/Fable split becomes structurally impossible rather than merely fixed.

5 · What it costs — honestly

Added

- **6 Convex tables**: 2 for sync (view heads, receipts), 4 for the execution rail (bindings, heads, boots, delivery attempts)
- **Client infra**: ~5k lines in `store/local-first/` (engine, persistence adapter, command runtime, contracts) — concentrated, branch-free, contract-tested
- **Daemon rail**: coordinator + drivers behind `CODECAST_FENCED_EXECUTION_V1`, currently off everywhere
- A transitional period where v1 and v2 both run (bounded per slice by an explicit deletion gate)

Deleted at the end (per slice, gated)

- Change-feed branches and timestamp catch-up crawls
- Ghost sweeps and repair timers
- Pending-field value-merge logic in the store
- Tombstone bookkeeping for synced collections
- Unjournaled raw dispatch paths
- The daemon's duplicated runtime-resolution fallbacks

The per-feature cost after last week's factoring pass: adding a synced view is a ~15-line client contract (types derived from the Convex query reference — shape drift is a compile error), one v2 query built from shared envelope

helpers, and a ~20-line writer binding. The engine itself contains zero per-view branches; the two reference contracts (buckets, comments) total 28 lines.

```
// the entire feature-side surface, live in the app today
const comments = useLocalView(commentsByConversationView, { conversationId });
// durable rows paint first (offline included); optimistic overlays are folded in;
// merging, coverage, epochs, retries, and persistence are not expressible here.
```

6 · The evidence discipline (how we know it's right)

The rollout is gated by proof rather than confidence, in four independent layers:

LAYER	WHAT IT GUARANTEES
Live digest equivalence (the centerpiece)	In shadow mode, both pipelines run for the same principal. On every quiescent state, the rows v1 currently renders are SHA-256-digested against the v2 durable view. Any divergence, on any field v1 delivers, is a loud console error plus a counter — checked continuously against real production data, by code that ships with the product (`__CODECAST_SHADOW_VALIDATION__()`). Payload-free: identities, row counts, one-way digests.
Deterministic transition coverage	~2,040 web+convex tests: the persistence adapter durability contract, stale-source and principal-epoch fencing, supersession interleavings, command journal transitions, FIFO application ordering, forbidden/missing view clears.
Structural impossibility	Client result types derive from server query references (drift = compile error). CI guards enforce the single-writer boundary. The engine has no per-view code to get wrong.
Fail-closed rollout	Flags are per-slice, default off, double-gated (a typo keeps everything off). In shadow, v1 stays authoritative. Rollback at any stage is reverting an env var. The server's authority is untouched throughout — the worst possible v2 failure is a mis-render, never corruption.

> **Evidence so far (as of Jul 29):** local shadow runs across buckets + three real comment threads — digest-equal, zero mismatches. Cutover exercised locally end-to-end: comment dock and labels rendered purely from the durable view with v1 feeds switched off, including a live comment create → durable materialization under its server id → delete → clean removal. Prod is now soaking in shadow mode (Railway build `index-DDZ1owip`); the first production samples are digest-equal with `mismatchedViews: 0`.

7 · Where it stands, and what's next

STAGE	STATUS
Phase 0 security (query authz closures, principal-scoped persistence)	Shipped
Sync substrate (view heads, receipts, typed writers, envelopes)	Live on the shared backend, coexisting with all legacy traffic
Client engine + reference slices (buckets, comments)	Shipped; **prod soaking in shadow**, v1 authoritative
Cutover of reference slices	Gated on a clean team-wide soak (days, not weeks); one env-var flip, instantly reversible
Fenced execution rail	Built and tested at unit level; never exercised live — needs a daemon run with the flag on and a driver to move one conversation onto the fenced protocol. This is the next substantial piece of work.
Legacy deletion (the payoff)	Explicitly gated behind proven cutover, per slice — "do not leave two correctness systems active indefinitely" is a tracked obligation, and the deletion list in §5 is the deliverable

8 · Where pushback is most useful

- **Collection classification** — which collections genuinely earn durable replication vs. staying query-owned. The doc keeps inbox, tasks, and docs query-owned until their contracts are normalized; that call deserves your read.
- **The execution rail's driver surface** — the tmux and app-server drivers encode assumptions about session lifecycle you know better than anyone.
- **Deletion sequencing** — which legacy path to delete first once a slice proves out, and what monitoring you'd want in the window where both exist.
- **Anything that smells like a second domain model.** The red-flag list exists to be invoked; if some part of this reads as replication reconstructing product semantics, that is precisely the review the design asks for.

Sources: `docs/architecture/local-first-sync-and-runtime-binding-design.md` (full design, invariants §16, validation plan §18, decisions §22) and `local-first-frp-restart-brief.md` (the original failure analysis and the rejected first implementation). Live evidence: any signed-in prod console → `__CODECAST_SHADOW_VALIDATION__()`.